UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Y

UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA

Learning Guide and Lab Manual

Digital Design with SystemVerilog and Gowin Tang Nano 9K FPGA

Student:

Altamira Cisneros Luis Gerardo

Professor:

Dr. Mauricio Alonso Sánchez

June 5, 2026

Contents

1 Introduction to the Work Environment

Digital systems design has evolved from the physical connection of logic gates to the use of Hardware Description Languages (HDLs) implemented on Field-Programmable Gate Arrays (**FPGAs**).

This learning guide uses the **Gowin Tang Nano 9K** development board, based on the low-cost GW1NR-9C chip. To interact with the outside environment, the board connects to an expansion module equipped with a **TM1638** integrated circuit (which manages 8 7-segment displays, 8 input buttons, and 8 indicator LEDs) and a **480x272 pixel LCD screen**.

All the material in this guide is based on the work of **Yuri Panchul** and his repository basics-graphics-music, adapted specifically for the Tang Nano 9K board and its expansion module.

The design flow consists of:

1. **Modeling:** Writing SystemVerilog code (‘.sv‘).
2. **Synthesis and P&R:** Translating the code into a physical gate netlist using the **Gowin IDE** software.
3. **Programming:** Loading the binary file (‘.fs‘) to the SRAM or Flash memory of the FPGA via JTAG.

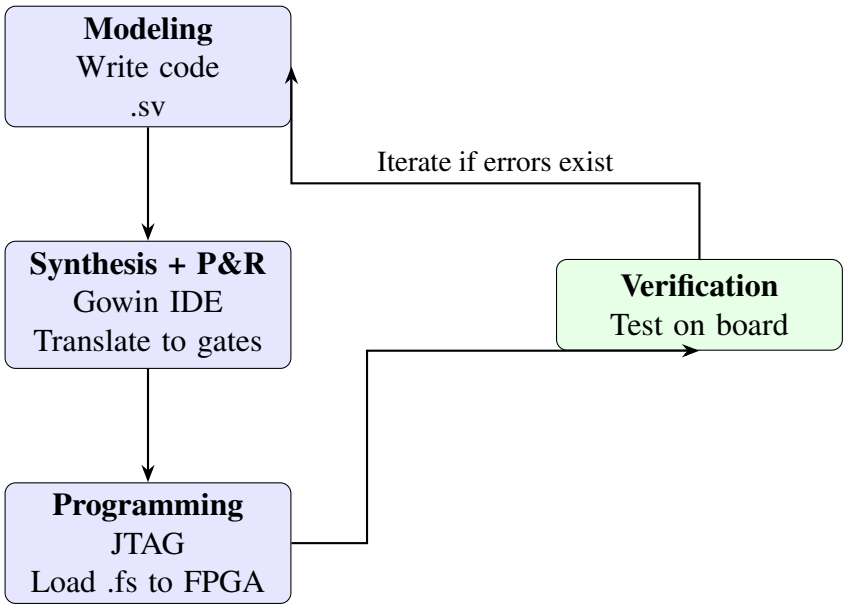


Figure 1: Digital design flow on FPGA.

2 Fundamental Concepts

2.1 What is an FPGA?

A **Field-Programmable Gate Array (FPGA)** is an integrated circuit that contains a matrix of configurable logic blocks, interconnected through a programmable routing network. Unlike a microcontroller that executes sequential instructions, an FPGA implements **physical digital circuits** described in a hardware description language. Programming it defines the logic functions, connections, and system architecture, achieving true parallelism and deterministic performance.

2.2 What is SystemVerilog?

SystemVerilog is a Hardware Description Language (HDL) that extends classical Verilog with advanced capabilities for modeling, verification, and synthesis. In this manual, it is used to describe the behavior of combinational and sequential circuits. Synthesis tools translate this code into the physical configuration of the FPGA.

2.3 The Tang Nano 9K Board

The **Gowin Tang Nano 9K** is a compact development board based on the GW1NR-9C FPGA, which includes:

- 8640 LUTs (look-up tables), sufficient for medium-complexity designs.
- Integrated 27 MHz oscillator.
- Connector for a 480x272 pixel LCD.
- GPIO pins for external modules (sensors, buttons, etc.).
- Programming via JTAG interface with included USB cable.

Its low cost and the availability of the free **Gowin IDE** development environment make it ideal for teaching and rapid prototyping.

2.4 Official Project Repository

All the source code, scripts, and examples in this manual are available on GitHub:

`https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K`

This repository is an adaptation of the original work by **Yuri Panchul** (`https://github.com/yuri-panchul/basics-graphics-music`), modified and optimized for the Gowin Tang Nano 9K board with LCD

and TM1638 expansion modules.

3 Development Environment Setup

Below are all the steps required to configure the development environment in **Ubuntu Linux** (22.04 LTS or higher). It is assumed that the reader has access to a terminal and an internet connection.

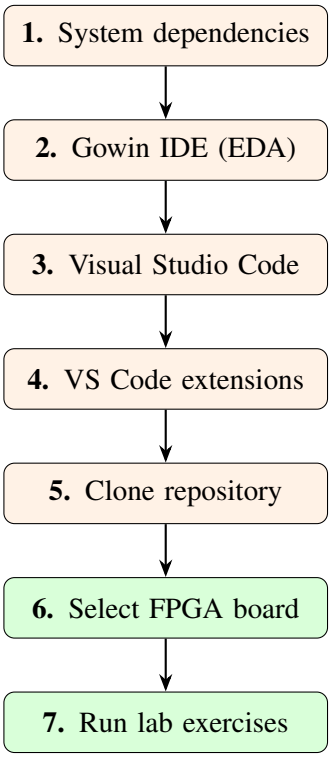


Figure 2: Summary of the installation and configuration process (7 steps).

3.1 Step 1: Installing System Dependencies

Open a terminal (`Ctrl+Alt+T`) and run the following commands to install the base tools:

Terminal

```
$ sudo apt update
$ sudo apt install git openfpgaloader
```

Explanation of each package:

- `git`: Version control system used to clone the repository.
- `openfpgaloader`: Open-source tool for programming FPGAs from various vendors (including Gowin) under Linux.

Note on USB permissions (udev rules)

To program the FPGA board without superuser (`sudo`) privileges, Linux requires udev rules. If you experience errors when detecting the board, add your user to the USB access group and reload the rules:

```
$ sudo usermod -aG plugdev $USER
$ sudo udevadm control --reload-rules && sudo udevadm trigger
```

Afterward, disconnect and reconnect the FPGA board's USB cable.

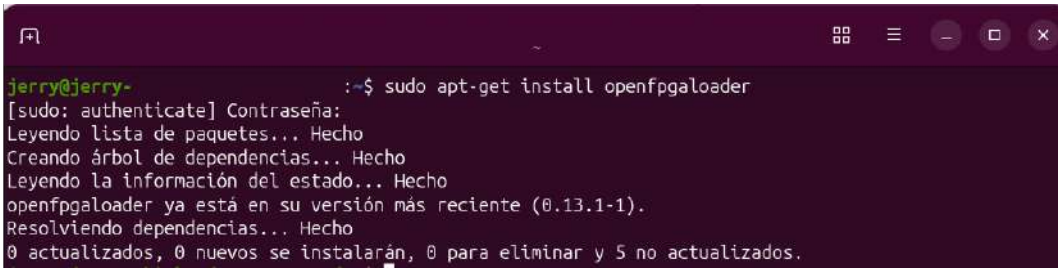


Figure 3: Installing Open FPGA Loader.

3.2 Step 2: Installing Gowin IDE (EDA)

The official Gowin software is required to synthesize designs and generate binary files for the FPGA.

1. **Create a Gowin account:** Visit `https://www.gowinsemi.com/en/support/download_eda/` and register for free.
2. **Download Gowin EDA:** Access the downloads section and select the education version for Linux: **Gowin V1.9 Education (Linux)**. Download the `.tar.gz` file.

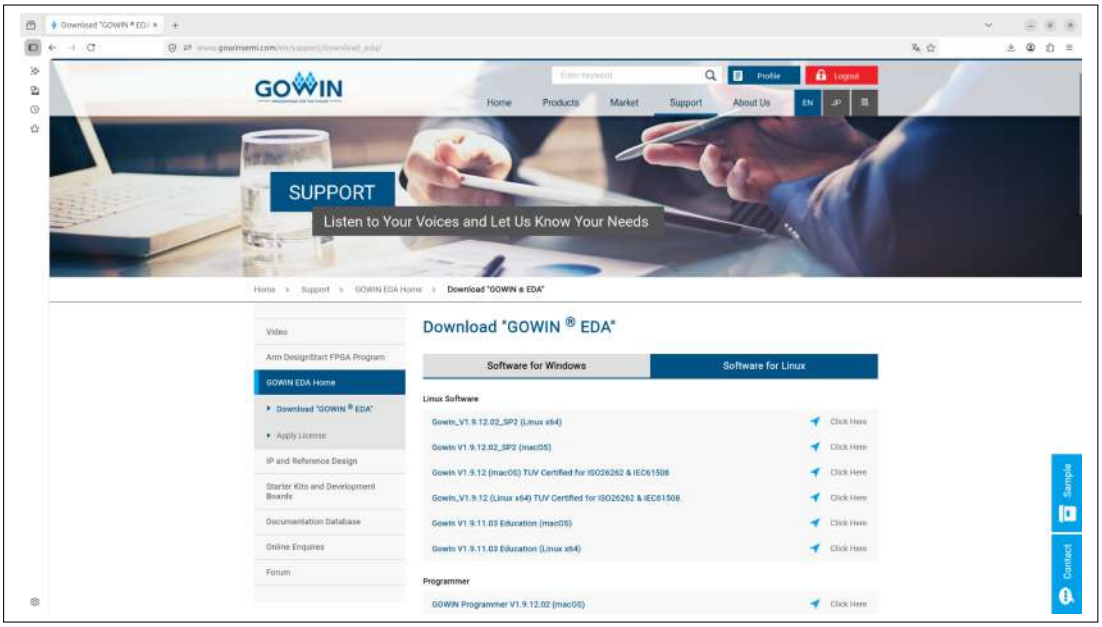


Figure 4: Gowin EDA download page.

3. **Extract the file:** In the terminal, navigate to the downloads folder and run:

```
Terminal
mkdir -p ~/gowin
tar -xf Gowin_V1.9.XX.XX_Education_Linux.tar.gz
-C ~/gowin --strip-components=1
```

4. **Verify installation:** The project scripts will automatically search for Gowin EDA in `~/gowin` or `/opt/gowin`.

3.3 Step 3: Installing Visual Studio Code

Although Gowin IDE includes its own editor, using **Visual Studio Code** is recommended due to its lightweight nature, extensions, and integrated terminal.

```
Terminal
$ sudo snap install code --classic
```

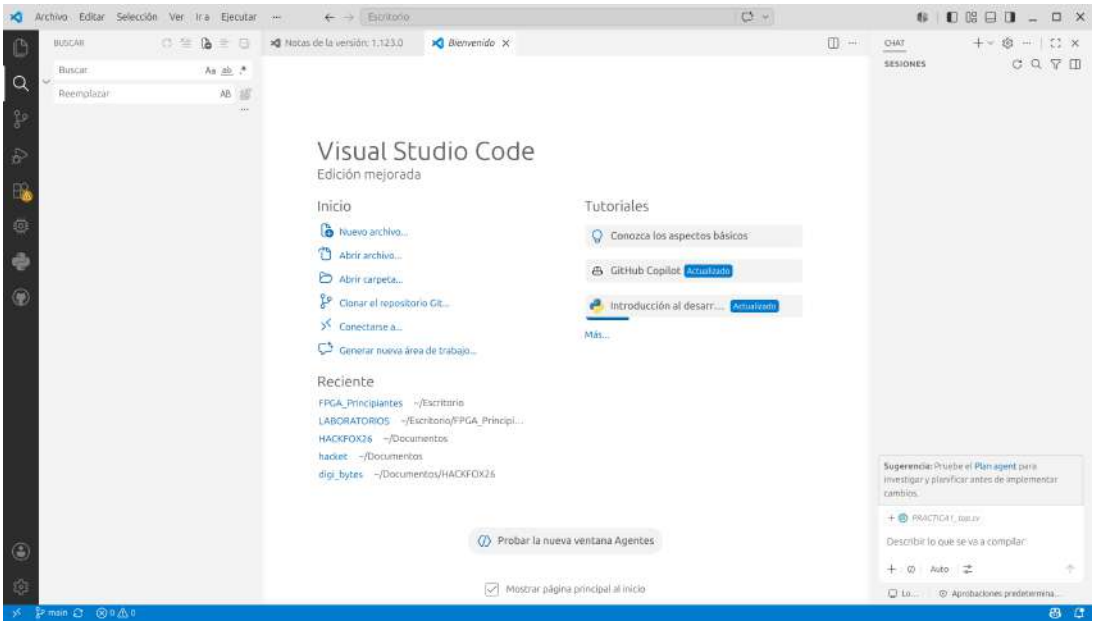


Figure 5: Visual Studio Code welcome screen.

3.4 Step 4: VS Code Extensions for SystemVerilog

To enable syntax highlighting, autocompletion, and code navigation, install the following extensions:

1. Open VS Code.
2. Go to the extensions section (`Ctrl+Shift+X`).
3. Search for and install:
    - **Verilog-HDL/SystemVerilog** (by `mshr-h`): Provides syntax coloring, autocompletion, and linter integration for Verilog and SystemVerilog.
    - **Verilog/SystemVerilog Tools** (by `eirikpre`): Helps with code navigation, module instantiation, and productivity utilities.

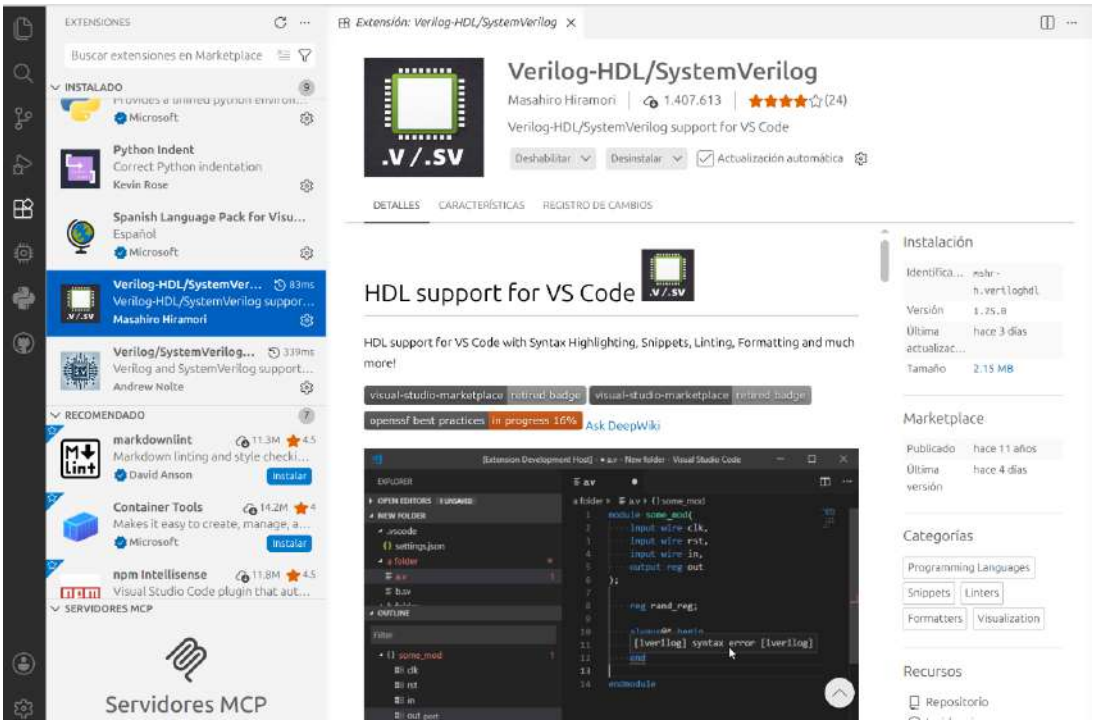


Figure 6: Recommended extensions for SystemVerilog in VS Code.

3.5 Step 5: Cloning the Project Repository

To obtain all the files for the labs:

1. Open a terminal in the folder where you want to download the project.
2. Run:

```
Terminal
    git clone https://github.com/JERRYALTAMIRA/
    FPGA_PRINCIPIANTES_TANG_NANO_9K.git
```

3. Enter the downloaded directory:

```
Terminal
    cd FPGA_PRINCIPIANTES_TANG_NANO_9K
```

4. To update the repository in the future:

```
Terminal
    git pull
```

4 SystemVerilog Fundamentals for FPGAs

Below is a summary of the basic concepts applied across all labs. It is recommended to read them before starting.

4.1 Top Module and Ports

In SystemVerilog, a design is structured into **modules**. The highest-level module (top) contains ports that connect physically to the FPGA pins. Each port is declared with a direction:

- `input logic` – signal entering the module (e.g., button, clock).
- `output logic` – signal leaving the module (e.g., LED, display segment).
- `inout logic` – bidirectional signal (use with caution).

Minimal example:

```
1        module practicas_top (
2        input  logic clock,
3        input  logic reset,
4        input  logic [7:0] key,
5        output logic [7:0] led
6        );
7        // circuit description
8        endmodule
9
```

4.2 Combinational Assignments (`assign`)

The `assign` statement creates a continuous connection, equivalent to physical wiring. It is used for combinational logic.

```
1        assign led[0] = ~key[0];    // LED turns on while the button is
   pressed
2
```

4.3 Sequential Logic (`always_ff`)

To register values (flip-flops), the `always_ff` block is used, sensitive to the rising edge of a clock (`posedge clock`) and an asynchronous reset signal.

```
        logic [7:0] counter;
        always_ff @(posedge clock or posedge reset) begin
        if (reset)        counter <= 8'h00;
        else              counter <= counter + 1;
        end

```

Assignments within `always_ff` must be non-blocking (<=) to ensure the correct inference of registers.

4.4 Control Structures

Inside `always` blocks, standard programming control structures can be used:

- `if-else` for conditions.
- `case` for multiple selection (widely used in decoders).
- `for` loops for generation (only inside `generate` blocks or for repetitive logic resolved at synthesis time).

Example with `case`:

```
        case (estado)
        2'b00: led = 8'b00000001;
        2'b01: led = 8'b00000010;
        default: led = 8'b00000000;
        endcase

```

4.5 Data Types

SystemVerilog introduces the `logic` type, which replaces traditional Verilog `wire` and `reg` types. For buses, the notation `[N:0]` (high:low index) is used. Example: `logic [7:0] data_bus;` (8 bits).

4.6 Operators

- Bitwise operators: `&`, `|`, `^`, `~`

- Logical operators: `&&`, `||`, `!`
- Concatenation: `{a, b}` joins signals.
- Arithmetic: `+`, `-`, `*` (synthesizable if size is limited).

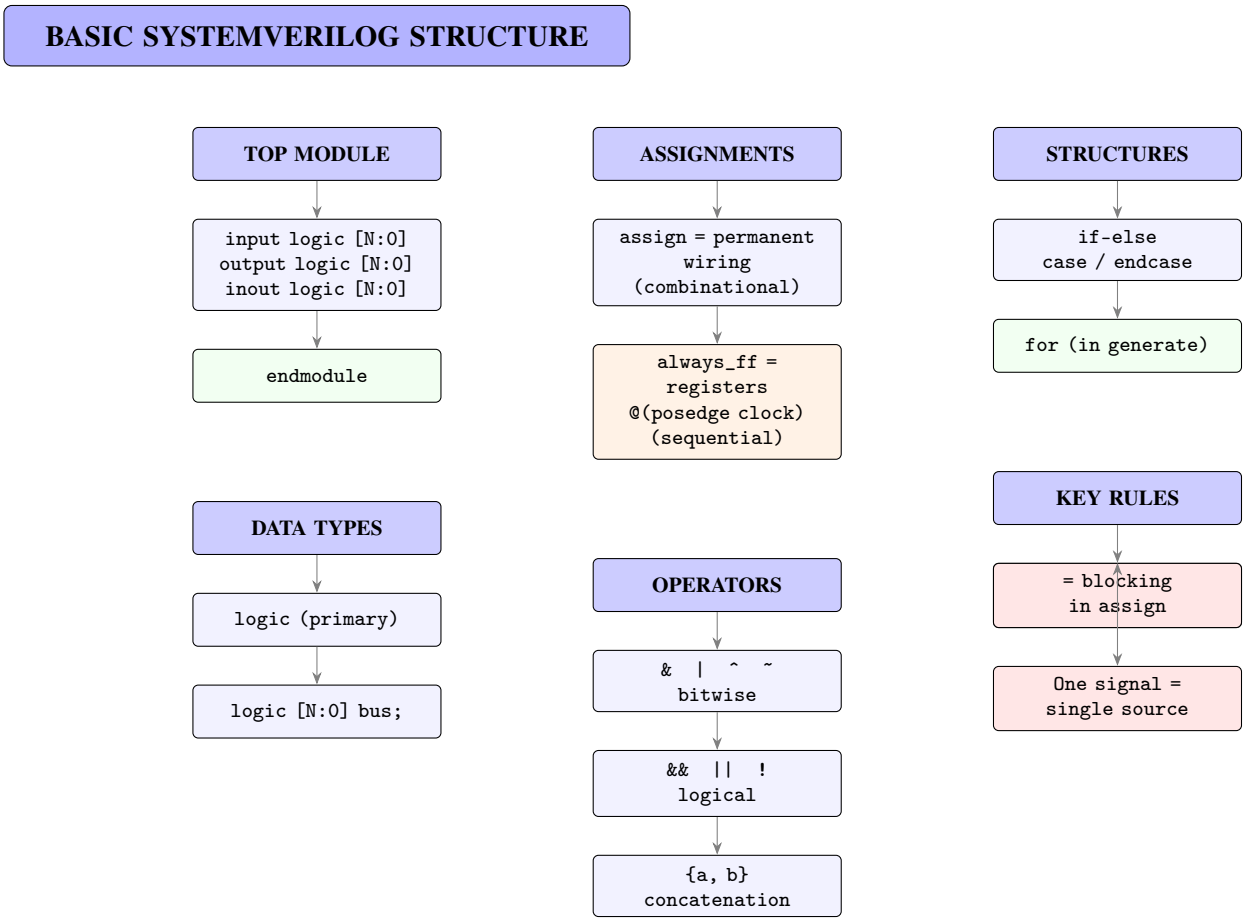


Figure 7: Visual summary of SystemVerilog fundamentals for FPGA.

With these fundamentals, the code in the following labs can be easily understood. It is recommended to consult the complete SystemVerilog documentation for deeper details.

5 Guide for Using and Running the Labs

Once the environment is set up, you can proceed to compile and load the designs onto the FPGA.

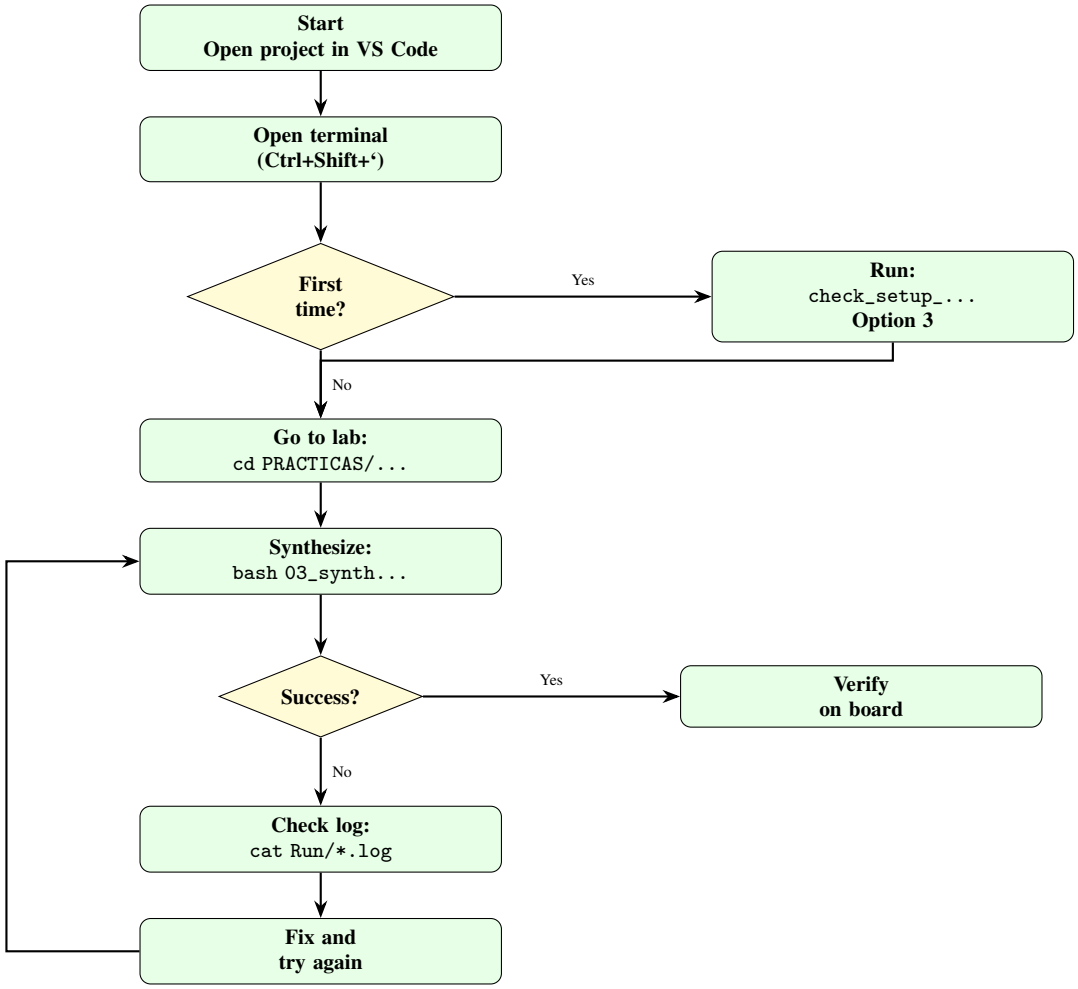


Figure 8: Flowchart for executing a lab exercise.

5.1 Step 1: Selecting the FPGA Board (First time only)

Before running any lab, you must specify which board you are using.

1. Open the project in VS Code: `File` > `Open Folder` and select the folder `FPGA_PRINCIPIANTES_TANG_NANO_9K`.

2. Open the integrated terminal in VS Code: `Terminal` > `New Terminal` or using the shortcut `Ctrl + Shift + ‘`.

3. In the integrated terminal (ensuring you are at the project root), run:

```
bash check_setup_and_choose_fpga_board.bash
```

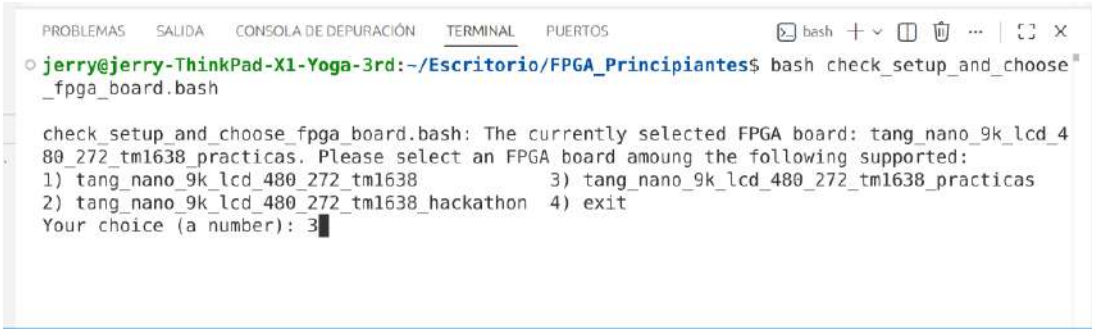


Figure 9: Starting the execution of the board selection script.

4. A list of supported boards will appear. Look for the option that says exactly (without the leading # symbol):

   `tang_nano_9k_lcd_480_272_tm1638_practicas`

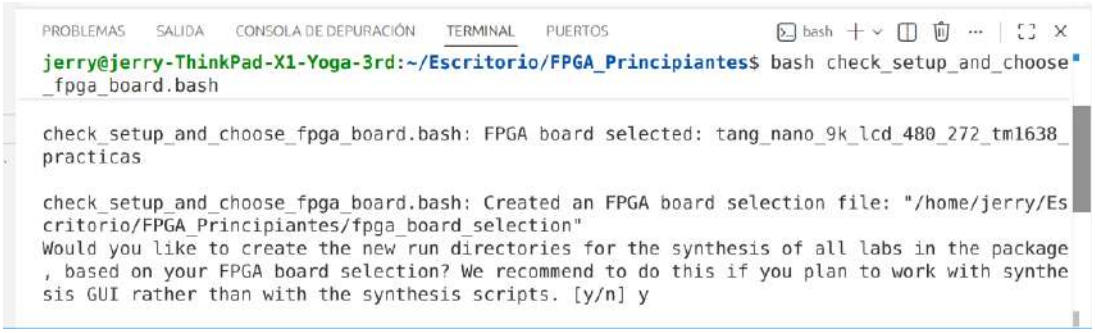


Figure 10: Selecting the Tang Nano 9K board in the interactive menu.

5. Type the corresponding number (usually `3`) and press `Enter`.

6. When asked if you want to use the Gowin GUI, select `N` (No) to continue with the command-line flow.

Important Note: Lines starting with # are comments and are disabled. Only select the line that does NOT start with a #.

5.2 Step 2: Running a Lab

Each lab is located in its own folder under the `PRACTICAS/` directory. To compile and load any lab:

1. In the integrated terminal of VS Code, navigate to the desired lab. For example, for Lab 1:

   ```
   cd PRACTICAS/PRACTICA1
   ```

2. Run the synthesis and programming script:

   ```
   bash 03_synthesize_for_fpga.bash
   ```

3. The script will automatically perform:

   (a) **Logic synthesis:** Translates SystemVerilog code into a gate netlist.

   (b) **Place & Route:** Maps the gates to physical locations on the chip.

   (c) **Bitstream generation:** Creates the `.fs` file in the `Run/` folder.

   (d) **Programming:** Loads the bitstream to the FPGA via JTAG.

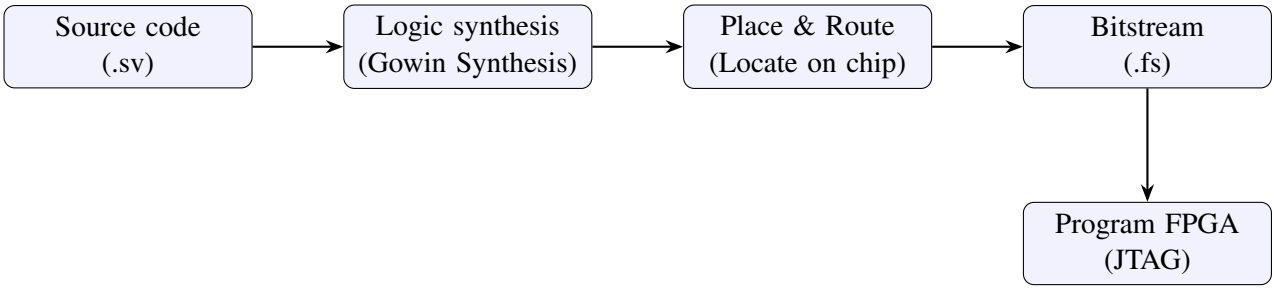


Figure 11: Internal process of the `03_synthesize_for_fpga.bash` script.

4. If synthesis is successful, a message similar to the following will be displayed:

Figure 12: Successful execution.

5.3 Other Useful Scripts

Script	Description
`03_synthesize_for_fpga.bash`	Synthesizes the design and programs the FPGA (main script).
`04_configure_fpga.bash`	Loads a previously compiled bitstream without re-synthesizing.
`05_run_gui_for_fpga_synthesis.bash`	Opens the Gowin IDE graphical user interface.
`01_clean.bash`	Deletes generated temporary files.

Table 1: Available scripts in each lab directory.

5.4 Summary of the Daily Workflow

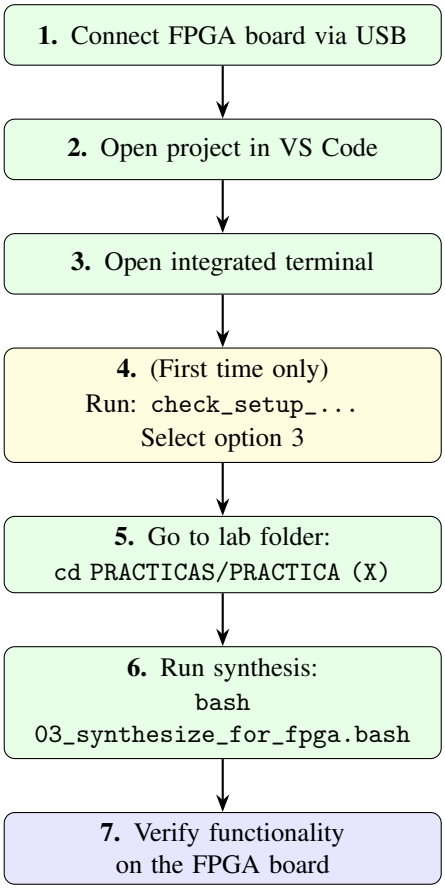


Figure 13: Daily workflow for lab exercises (7 steps).

6 Troubleshooting Common Issues

Error	Solution
`gw_sh: command not found`	Gowin IDE is not installed or not in the expected path.
`FPGA not detected`	Check the USB cable, udev rules (Step 1), and reconnect the board.
`Multiple drivers`	The same signal is being assigned in more than one place in the code.
Compilation error	Review the log: `cat Run/*.log | grep -i error`

Table 2: Common issues and their solutions.

7 Laboratory Exercises

7.1 Lab 1: Static LED Control

Objective: Familiarize with SystemVerilog syntax and the Gowin environment by making a direct and unconditional physical connection.

Goals: Turn on indicator LED 0 on the board and keep the other 7 LEDs permanently off.

Key Concept (Continuous Assignment - `assign`): In hardware design, an `assign` statement represents a permanent weld or connection. The signal on the right side is continuously connected to the left side of the operator.

```
1         // Only the LED assignment is shown; unused signals are set to 0.
2         assign led = 8'b00000001;   // bit0 = 1, all others = 0
3
```

Listing 1: Essential snippet for Lab 1

GitHub Repository: `https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA1/PRACTICA1_top.sv`

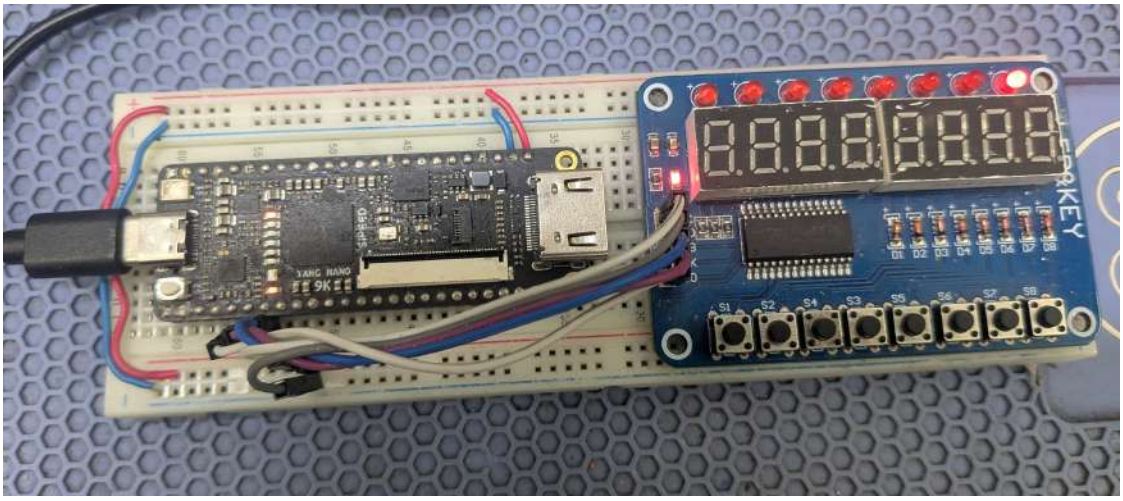


Figure 14: Result of physically turning on the LED.

7.2 Lab 2: Button-Controlled LED

Objective: Read physical inputs in real-time and connect them to the FPGA's logical outputs.

Goals: Control LED 0 through the press of Button 0.

Key Concept (Negative Logic and Concatenation): Many development buttons connect with pull-up resistors, meaning the pin reads a high state (`1'b1`) when not pressed, and a low state (`1'b0`) when pressed (connection to ground). For the output to respond intuitively (Button pressed = LED on), the signal must be inverted using the NOT operator (`~`).

```
        // Assign the LED bus using concatenation:
        assign led = {7'b0000000, ~key[0]};
        // Upper 7 bits set to 0, bit 0 follows the inverted button state.

```

Listing 2: Essential snippet for Lab 2

GitHub Repository: `https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA2/PRACTICA2_top.sv`

7.3 Lab 3: Basic Logic Gates

Objective: Implement elementary combinational Boolean logic functions.

Goals: Evaluate AND, OR, and XOR gates using two buttons and three LEDs as outputs.

Key Concept (Bitwise Operators vs. Logical Operators): In HDLs, bitwise operators (`&`, `|`, `^`, `~`) act individually on each physical bit of the signal. Logical operators (`&&`, `||`, `!`) return a conditional boolean value (true/false) for flow control. In combinational logic, bitwise operators must always be used.

```
        // Convert buttons to active-high logic
        logic b0, b1;
        assign b0 = ~key[0];
        assign b1 = ~key[1];

        // Gates
        assign led[0] = b0 & b1;    // AND
        assign led[1] = b0 | b1;    // OR
        assign led[2] = b0 ^ b1;    // XOR

```

Listing 3: Essential snippet for Lab 3

GitHub Repository: `https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA3/PRACTICA3_top.sv`

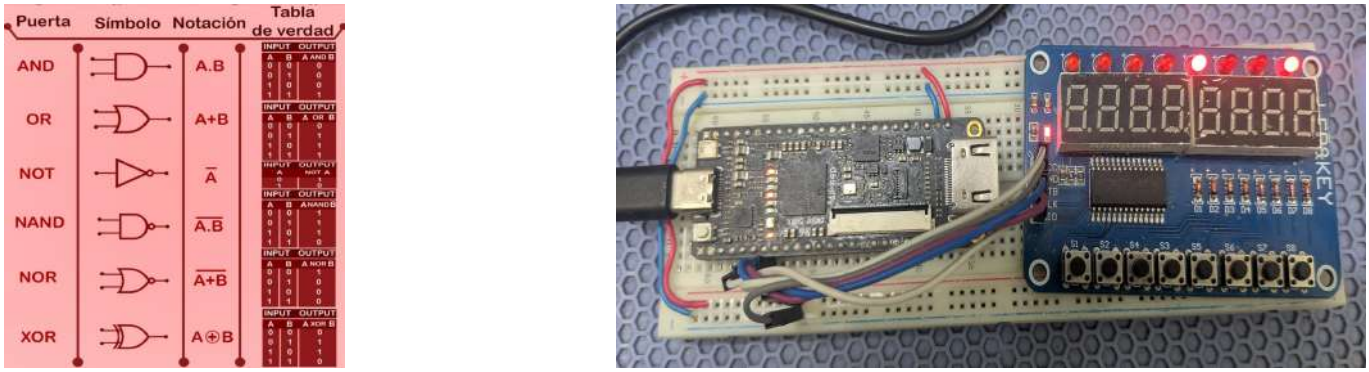


Figure 15: Physical test of logic gates.

7.4 Lab 4: Blinking LED

Objective: Introduce sequential logic and clock frequency division.

Goals: Make an LED blink at a visible frequency (0.8 Hz) derived from the board's 27 MHz master clock.

Key Concept (Sequential Logic and Counter-Based Division): Sequential assignments are executed on the rising edge of a clock (`posedge clock`) inside an `always_ff` block. By adding `1` to a 25-bit binary counter on each clock cycle, each bit acts as a frequency divisor by powers of 2:

$$\text{Frequency of bit } N = \frac{f_{clk}}{2^{N+1}}$$

For bit 24 (MSB): $\frac{27\,\text{MHz}}{2^{25}} \approx 0.8\,\text{Hz}$ (a full period of 1.24 seconds).

```
        logic [24:0] counter;
        always_ff @(posedge clock or posedge reset) begin
        if (reset) counter <= 25'b0;
        else       counter <= counter + 25'b1;
        end
        assign led[0] = counter[24];  // the most significant bit blinks

```

Listing 4: Essential snippet for Lab 4

GitHub Repository: `https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA4/PRACTICA4_top.sv`

7.5 Lab 5: 8-Bit Binary Counter

Objective: Observe binary representation of data and rapid sequential incrementing.

Goals: Display a continuous binary count from 0 to 255 on the 8 LEDs using a slow clock.

Key Concept (Slow Clock - `slow_clock`): A fast counter running at 27 MHz is impossible to observe with the naked eye if connected directly to the LEDs. The board provides a slow clock signal (`slow_clock`) of approximately 100 Hz, allowing the bit transitions on the LEDs to be easily observed.

```
        logic [7:0] counter;
        always_ff @(posedge slow_clock or posedge reset) begin
        if (reset) counter <= 8'h00;
        else       counter <= counter + 8'h01;
        end
        assign led = counter;

```

Listing 5: Essential snippet for Lab 5

GitHub Repository: `https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA5/PRACTICA5_top.sv`

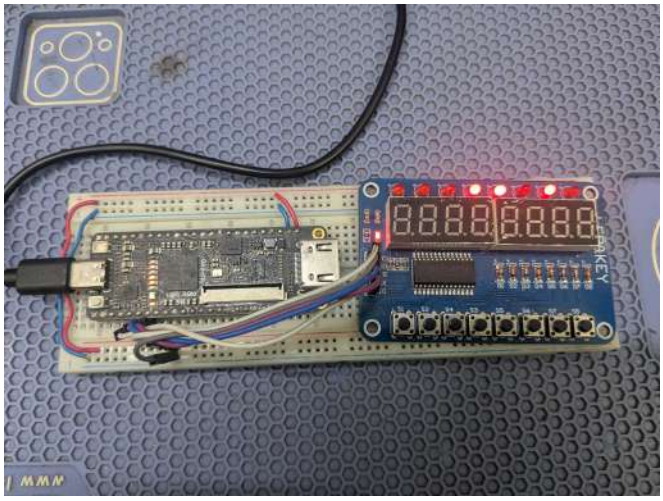


Figure 16: 8-bit binary counting.

7.6 Lab 6: 7-Segment Display (Single Digit)

Objective: Learn to decode binary numerical values into LED combinations for human readability.

Goals: Create a decimal counter from 0 to 9 and show it on the first digit of the 7-segment display.

Key Concept (7-Segment Bit Mapping): The board's display uses common cathode logic (1 = segment on). The bit order of the `abcdefgh` bus delivered to the board from right to left is: `{h, g, f, e, d, c, b, a}`, where `a` is the LSB (top segment) and `h` is the MSB (decimal point).

```
        function automatic [7:0] seg7_decode(input [3:0] num);
        case (num)
        4'h0: seg7_decode = 8'b00111111;  // 0
        4'h1: seg7_decode = 8'b00000110;  // 1
        // ... remaining digits
        default: seg7_decode = 8'b00000000;
        endcase
        endfunction

```

Listing 6: Essential snippet for Lab 6 (decoder)

GitHub Repository: `https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA6/PRACTICA6_top.sv`

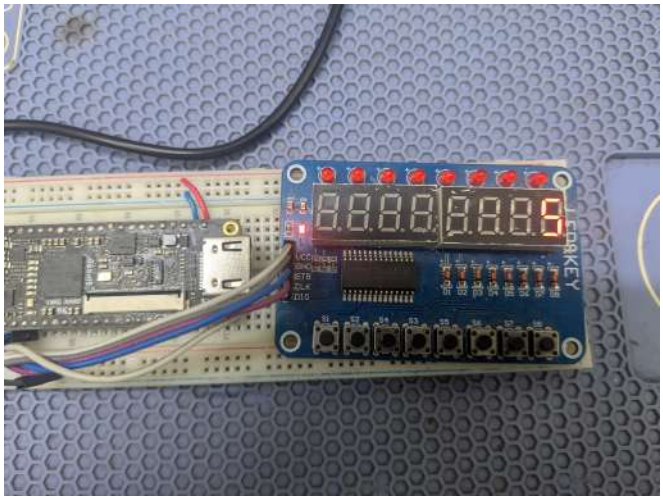


Figure 17: 7-segment display visualization.

7.7 Lab 7: Display Multiplexing and Debouncing

Objective: Display multi-digit numbers and filter mechanical bounces from buttons.

Goals: Increment a decimal counter from 0 to 255 upon pressing Button 0 and display it on three multiplexed digits of the 7-segment display.

Key Concept (Multiplexing and Debouncing):

- **Debouncing:** Physical buttons vibrate mechanically when pressed. A digital circuit must register the state change only when the input signal remains stable for several milliseconds.
- **Multiplexing:** To avoid duplicating wiring, digit data is sent in a fast sequence (500 Hz). The human eye perceives all digits as simultaneously active due to persistence of vision.

```
        // Stable edge detection
        wire b0_edge = b0_stable & ~b0_prev; // clean rising edge
        // Increment counter only on this edge
        always_ff @(posedge slow_clock or posedge reset) begin
        if (reset) counter <= 0;
        else if (b0_edge) counter <= counter + 1;
        end

```

Listing 7: Debouncing snippet (Lab 7)

GitHub Repository: `https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA7/PRACTICA7_top.sv`

Figure 18: 3-digit display multiplexing.

7.8 Lab 8: LED Traffic Light (State Machine)

Objective: Design structured sequential control systems.

Goals: Control three LEDs (red, yellow, green) emulating the cyclical and timed sequence of a real traffic light.

Key Concept (FSM - Finite State Machine): State machines divide the circuit's behavior into defined logical states. Using enumerated data types (`typedef enum`) avoids using direct or "magic" numerical values, improving code readability and enabling compiler optimizations.

```
1        typedef enum logic [1:0] { ROJO, ROJO_AMARILLO, VERDE, AMARILLO }
   estado_t;
2        estado_t estado;
3        always_ff @(posedge slow_clock) begin
4        case (estado)
5        ROJO:          if (cuenta == 0) estado <= ROJO_AMARILLO;
6        ROJO_AMARILLO: if (cuenta == 0) estado <= VERDE;
7        // ... transitions
8        endcase
9        end
10
```

Listing 8: FSM snippet (Lab 8)

GitHub Repository: `https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA8/PRACTICA8_top.sv`

7.9 Lab 9: Introduction to LCD Graphics (Coordinates and Colors)

Objective: Understand the combinational operation of a graphics generator on a digital screen.

Goals: Draw three static rectangles of different colors (red, green, blue) at specific positions on the LCD screen.

Key Concept (LCD Resolution and Coordinates): The development kit's screen has an active resolution of **480x272 pixels**. The combinational block constantly receives the coordinates x (0-479) and y (0-271) of the pixel currently being drawn by the screen, and must decide what color to assign to the output in parallel.

```
        always_comb begin
        red = 5'h00; green = 6'h00; blue = 5'h00;  // black background
        if (x > 50 && x < 150 && y > 50 && y < 100) begin
        red = 5'h1F; green = 6'h00; blue = 5'h00;  // red
        end
        // ... other rectangles
        end

```

Listing 9: Combinational drawing snippet (Lab 9)

GitHub Repository: https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA9/PRACTICA9_top.sv

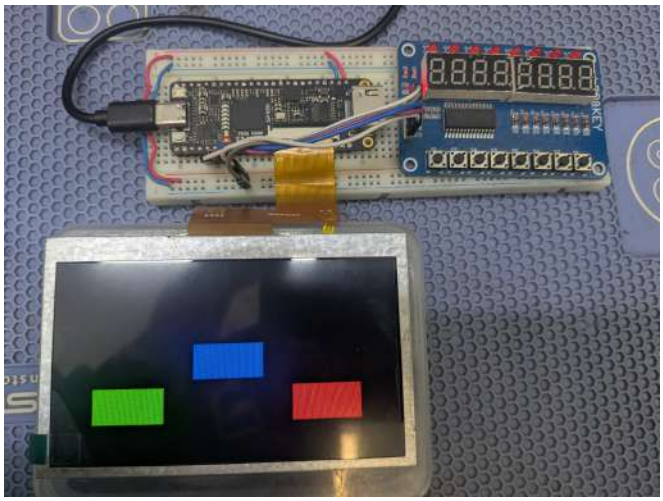


Figure 19: Static graphics on LCD screen.

7.10 Lab 10: Automatic Movement on LCD Screen

Objective: Synchronize logical events with the video frame refresh rate.

Goals: Create an animation where a yellow rectangle moves horizontally across the screen from left to right indefinitely.

Key Concept (Synchronization Signals and Frames Per Second): Movement requires updating the figure's position exactly once per video frame (typically 30 or 60 Hz). The `strobe_gen` module generates a single-cycle pulse at the desired frequency to update the figure's position register.

```
1         logic enable;
2         strobe_gen #(.clk_mhz(27), .strobe_hz(30)) i_strobe (.clk(clock), .
    rst(reset), .strobe(enable));
3         always_ff @(posedge clock) begin
4         if (enable) begin
5         if (pos_x >= 400) pos_x <= 0;
6         else              pos_x <= pos_x + 1;
7         end
8         end
9
```

Listing 10: Animation snippet (Lab 10)

GitHub Repository: `https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA10/PRACTICA10_top.sv`

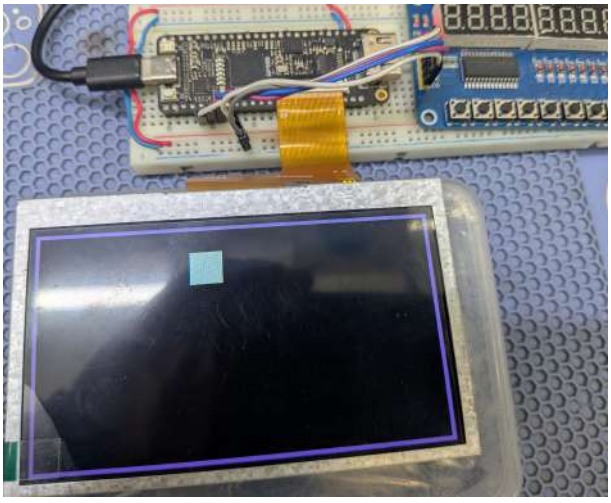


Figure 20: Horizontal object animation.

7.11 Lab 11: Interactive Control via Buttons

Objective: Combine asynchronous input reads with dynamic graphics generation.

Goals: Move a white square in two dimensions (up, down, left, right) using four buttons on the development board.

Key Concept (Two-Dimensional Control and Interfaces): Asynchronous button inputs are mapped to local direction variables. On each slow clock cycle, if a button is active, the coordinate of the figure is modified, validating strict geometric boundaries so the object does not disappear from the active screen area.

```
        logic left, right, up, down;
        assign left = ~key[0]; assign right = ~key[1];
        assign up   = ~key[2]; assign down  = ~key[3];
        always_ff @(posedge slow_clock) begin
        if (left  && pos_x > 5)   pos_x <= pos_x - 5;
        if (right && pos_x < 425) pos_x <= pos_x + 5;
        // ... vertical movement
        end

```

Listing 11: Control snippet (Lab 11)

GitHub Repository: `https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA11/PRACTICA11_top.sv`

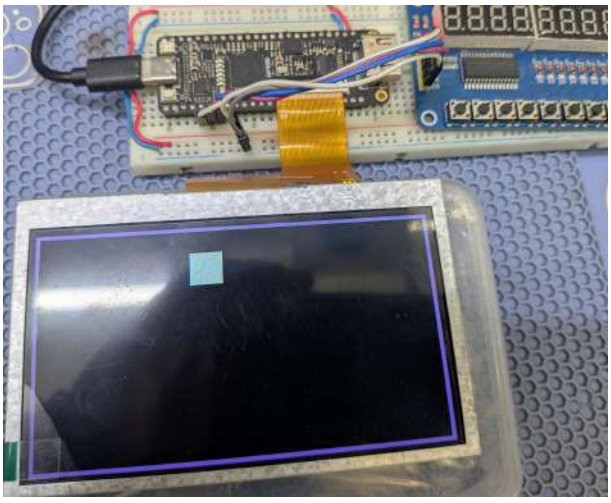


Figure 21: Object animation controlled by buttons.

7.12 Lab 12: Bouncing Ball (LCD Physics Animation)

Objective: Implement simple physics simulation algorithms in hardware.

Goals: Simulate the continuous movement and bounce of a red circular ball against the physical boundaries of the 480x272 pixel screen.

Key Concept (Symmetric Collision Boundaries): For a circular object with radius $R = 20$ pixels, the ball collides and must bounce (invert its velocity direction `dir`) when the ball's center touches the adjusted physical boundaries: * Left Boundary: $X \leq 20$. Right Boundary: $X \geq 460$ $(480 - 20)$. * Top Boundary: $Y \leq 20$. Bottom Boundary: $Y \geq 252$ $(272 - 20)$.

```
        always_ff @(posedge clock) begin
        if (dir_x == 0) begin
        if (pos_x >= 460) dir_x <= 1; else pos_x <= pos_x + 4;
        end else begin
        if (pos_x <= 20)  dir_x <= 0; else pos_x <= pos_x - 4;
        end
        // analogous for dir_y
        end

```

Listing 12: Physics snippet (Lab 12)

GitHub Repository: `https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA12/PRACTICA12_top.sv`

Figure 22: Elastic collision physics on LCD.

7.13 Lab 13: Animated Traffic Light on LCD

Objective: Combine logical state machines with complex video renderers.

Goals: Draw a traffic light silhouette on screen with three circular lamps (red, yellow, and green) that turn on cyclically, synchronized with an internal FSM.

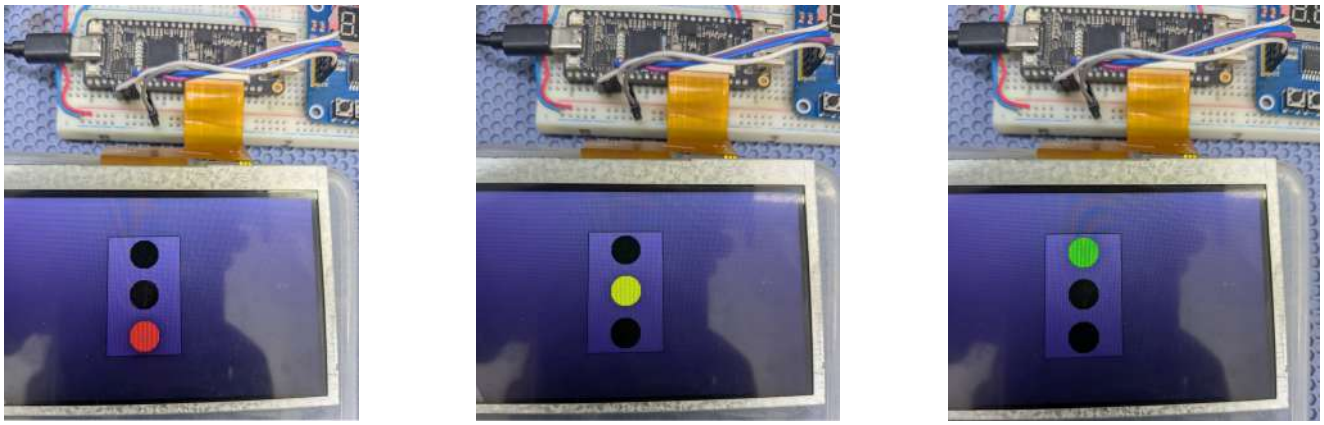
Key Concept (Non-Overlapping Geometric Sizing): To achieve a clean graphical design, dimensions of the drawn elements must be mathematically calculated. If a traffic light is centered at $Y = 136$ with three lamps of radius $R = 20$ and a vertical center spacing of 55 pixels, the lights will be rendered at $Y_{red} = 81$, $Y_{yellow} = 136$, $Y_{green} = 191$ without overlapping, fitting perfectly inside the gray frame box.

```
        localparam CENTRO_X = 240, CENTRO_Y = 136, RADIO = 20, SEP = 55;
        localparam ROJO_Y = CENTRO_Y - SEP, AMARILLO_Y = CENTRO_Y, VERDE_Y
   = CENTRO_Y + SEP;
        always_comb begin
        // body and light drawings conditioned on the FSM state
        end

```

Listing 13: LCD traffic light snippet (Lab 13)

GitHub Repository: `https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA13/PRACTICA13_top.sv`



Red light Yellow light Green light

Figure 23: Graphical traffic light synchronized with FSM: red, yellow, and green phases.

7.14 Lab 14: Radar and Ultrasonic Distance Measurement

Objective: Integrate third-party peripherals, perform mathematical scale conversion, and implement alarm control with visual warnings.

Goals: Read distance data from an HC-SR04 sensor, display the measurement in real centimeters on the 7-segment display, and turn on alarm LED 0 if an object is closer than 30 cm.

Key Concept (Signal Contention Resolution and Physical Conversion):

- **Multiple Driver:** If a physical pin is declared with a direct `assign` to zero and is simultaneously assigned the output of a sub-module, the compiler will throw an error. A single source of excitation must be ensured for each port.
- **Physical Conversion from Ticks to Centimeters:** Sound travels at 343 m/s. At a frequency of 27 MHz, the sensor counts clock cycles (`echo_cnt`). Since the signal travels back and forth:

$$\text{Cycles per Centimeter} = \frac{27 \times 10^6\,\text{Hz} \times 2}{343 \times 100} \approx 1574\,\text{cycles/cm}$$

Since the sensor sub-module delivers a scaled measurement of `distance = echo_cnt / 16`, the real distance in centimeters is:

$$\text{Distance (cm)} = \frac{\text{distance} \times 16}{1574} \approx \frac{\text{distance}}{98}$$

```
        wire [15:0] distance;
        ultrasonic_distance_sensor #(.clk_frequency(27_000_000)) i_sensor
    (...);
        assign distance_cm = distance / 16'd98;
        always_comb begin
        led = 0;
        if (distance_cm < 30) led[0] = 1'b1; // alarm
        end

```

Listing 14: Measurement snippet (Lab 14)

GitHub Repository: `https://github.com/JERRYALTAMIRA/FPGA_PRINCIPIANTES_TANG_NANO_9K/blob/main/PRACTICAS/PRACTICA14/PRACTICA14_top.sv`

8 Extending the Project: Adding Peripherals

Once the basic labs are completed, the system can be expanded to include other peripherals or communication protocols. Below are general guidelines.

8.1 Adding a Custom Module

Each peripheral should be encapsulated in an independent SystemVerilog module. For example, to add an I2C temperature sensor:

1. Write the module (e.g., `i2c_master.sv`) respecting the system's clock and reset signals.
2. Instantiate the module in the top module:

```
            i2c_master #(.CLK_FREQ(27_000_000)) i_i2c (
            .clk(clock), .rst(reset), .sda(gpio[2]), .scl(gpio[3]), .
   data(temp_data)
            );

```

3. Connect the data signals to the logic of visual display (LCD or 7-segment display).
4. Modify the physical constraints file (`.cst`) to assign the corresponding GPIO pins.

8.2 Pin Mapping

Review the schematic of the Tang Nano 9K and the expansion module to select available pins. Pin assignment is performed in the **Gowin IDE** through the constraint editor or by directly editing the `.cst` file. Example:

```
IO_LOC "gpio[0]" 10, 3, 1;  // physical pin
IO_PORT "gpio[0]" PULL_MODE=UP;
```

8.3 Automated Synthesis

Update the `03_synthesize_for_fpga.bash` script (or the TCL file) to include the new source files. This ensures that executing the script synthesizes the entire project consistently. The GitHub repositories include examples of these scripts.

8.4 Recommendations

- Test each peripheral separately before integrating it into the complete system.
- Use simulations (e.g., with ModelSim or GHDL) to verify logic before loading it onto the FPGA.
- Document the peripheral description and any changes in the GitHub repository.

9 Design Recommendations and Best Practices

When designing digital systems on an FPGA using SystemVerilog, it is essential to follow strict rules to ensure that the synthesized circuit operates reliably in hardware:

1. **Avoid Multiple Drivers:** A signal can only be assigned within a single `always` block or a single `assign` statement. Connecting multiple sources to a signal creates electrical contention conflicts and critical compiler errors.

2. **Fully Synchronous Design:** Always use the master clock (`clock`) as the clock signal in sequential `always_ff` blocks. For slower speeds, use clock enables instead of creating clocks divided by combinational logic, which increases clock skew.

3. **Signal Assignment:**

   - Use non-blocking assignment (<=) inside sequential `always_ff` blocks.
   - Use blocking assignment (=) inside combinational `always_comb` blocks.
   - Never mix both types of assignments in the same block.

4. **Bus Declaration:** Use SystemVerilog's `logic` type instead of traditional Verilog `wire` and `reg` types. The `logic` type automatically prevents accidental double assignments during synthesis analysis.

10 Conclusions

This lab manual serves as a progressive educational scaffolding for any student, starting from scratch, to assimilate theoretical concepts of digital electronics and implement them physically on the Tang Nano 9K FPGA. Incremental learning—from turning on a simple LED to processing and scaling digital signals from ultrasonic sensors—equips the student with the practical skills indispensable in modern hardware design.

11 Equations Appendix

Speed of sound equation for the ultrasonic sensor:

$$d = \frac{v \times t}{2}$$

Where: * d: Distance in meters. * v: Speed of sound in air ($\approx 343\,\text{m/s}$). * t: Elapsed echo time in seconds.

Frequency division relationship for a binary counter:

$$f_{\text{out}} = \frac{f_{\text{in}}}{2^{N+1}}$$